

Can Depth Features Detect Objects? ZipDepth as a Backbone for YOLO26n

Lab entry task report · Amirmahdi · September 2026 · KITTI 2D detection, Google Colab (Tesla T4)

Summary

- **Question.** Are features learned for monocular depth estimation useful for object detection? YOLO26n is the baseline (**Run A**); its backbone is replaced by the depth-pretrained ZipDepth-base encoder while the YOLO26 neck and detection head are kept (**Run B**). Both are fine-tuned on KITTI with identical settings.
- **Accuracy.** mAP50-95 drops from **0.588 to 0.472** (−0.116) and mAP50 from 0.849 to 0.731. Run B is lower on all eight classes and all object sizes; the deficit is mainly *missed* objects, not class confusion.
- **Cost.** 3.0× parameters (2.38 M → 7.14 M), 3.35× GFLOPs, 0.62× throughput at batch 32 and +17 % training memory. Batch-1 latency on the T4 is unchanged (≈10–16 ms) because it is dominated by kernel-launch overhead.
- **Depth hypothesis.** Depth features did not help small / distant objects: small-car AP50-95 falls 0.550 → 0.470 (−15 % relative) versus −5 % for large cars.
- **Open point.** Run B had not converged at 50 epochs, and the optional run with a randomly initialised encoder (Run C) has not been trained yet, so the effect of depth pretraining itself is not isolated.

1. Experimental design

The study is a controlled comparison in which only the backbone differs. Data, input size, schedule, optimiser, seed, neck/head initialisation and evaluation code are shared, and both models are evaluated back to back in one session.

Run	Backbone (layers 0–10 of YOLO26n)	Neck + head	Status
A	YOLO26n backbone, COCO-pretrained	YOLO26n, COCO-initialised	done
B	ZipDepth-base encoder, depth-pretrained (zipdepth_base.pth) + 1×1 adapters	YOLO26n, COCO-initialised	done
C (optional)	same as B, encoder randomly initialised	YOLO26n, COCO-initialised	not run

Setting	Value	Setting	Value
Dataset	Ultralytics KITTI: 5,985 train / 1,496 val, 8 classes	Optimiser	SGD, lr0 0.01 → 1e-4 (linear), momentum 0.937, wd 5e-4
Input size	640 (letterbox)	Warm-up / mosaic off	3 epochs / last 10 epochs
Epochs / batch	50 / 16 (nbs 64 → 4-step accumulation)	Other	AMP, seed 0, deterministic, no early stopping
Hardware	Colab, Tesla T4 (15 GB)	Software	torch 2.11 + CUDA 12.8, ultralytics 8.4.16x

2. Backbone feature maps and feature selection (tasks 1-2)

YOLO26n's neck (an FPN/PAN) reads three backbone tensors at strides 8, 16 and 32. The ZipDepth encoder produces a pyramid at the same strides, so its intermediate outputs can be plugged into the neck without changing any neck layer. Shapes below are for a 640×640 input.

Level	YOLO26n backbone (replaced)	ZipDepth encoder feature used	Why this tensor
P3, /8	layer 4 · C3k2 · 128×80×80	s2 (stage 2 output) · 96×80×80	finest scale; stage 2 includes multi-scale depthwise conv and strip pooling
P4, /16	layer 6 · C3k2 · 128×40×40	s3' · 192×40×40	stage 3 (SE + global context) after the encoder's cross-scale fusion with s4

Level	YOLO26n backbone (replaced)	ZipDepth encoder feature used	Why this tensor
P5, /32	layer 10 · C2PSA · 256×20×20	s4' · 384×20×20	stage 4 after the encoder's own SPPF and cross-scale fusion; replaces YOLO's SPPF + C2PSA

- Not used: the stride-2 and stride-4 stem features (24 / 48 channels) and the depth decoder (0.15 M parameters); the encoder is 6.63 M parameters (97 % of ZipDepth).
- Interface details found while studying the code: ImageNet normalisation sits *outside* the ZipDepth encoder and must be re-added; encoder activations are about 10× smaller than YOLO's (std ≈ 0.02 vs 0.5), which motivates BatchNorm in the adapters; s3' / s4' contain negative values (no activation after the cross-scale fusion).
- Analysis of the pretrained features (before fine-tuning) showed that ZipDepth channels track depth far more strongly than YOLO's (top-10 |r| with depth at P4 / P5: 0.65 / 0.88 vs 0.42 / 0.65), but in street scenes depth is largely explained by the image row; after removing the row trend the extra depth signal is mainly at P4 (0.41 vs 0.25). At the same time, region descriptors were less separable (cosine similarity between object / road / building regions 0.47–0.66 vs 0.25–0.36), an early hint of weaker class discrimination.

3. Modified model (task 3)

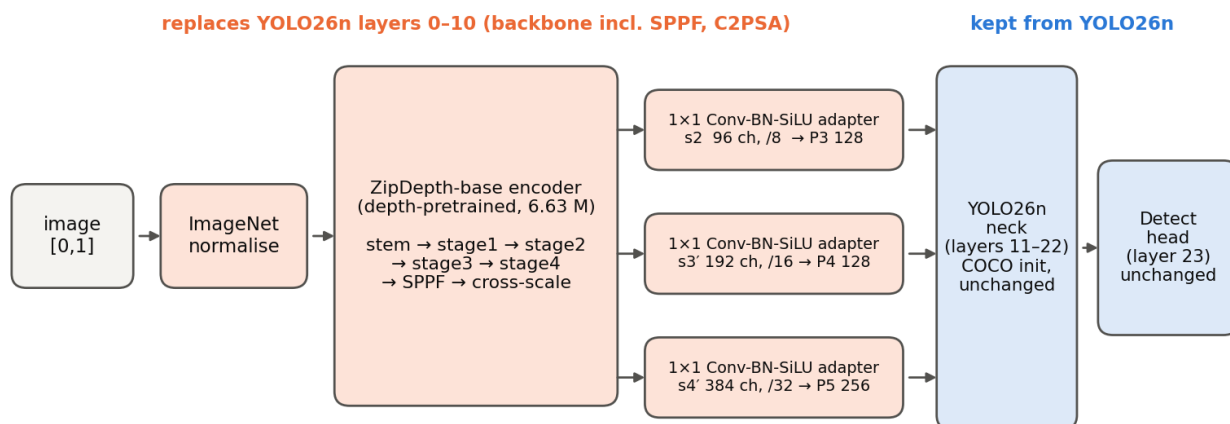


Figure 1. Run B. YOLO26n layers 0–10 are replaced by a single module (normalisation + ZipDepth encoder + three 1×1 Conv-BN-SiLU adapters); layers 4 / 6 / 10 become selectors of P3 / P4 / P5 and the other backbone indices become pass-through placeholders, so every layer reference of the unchanged neck stays valid.

The model is built in **zipdepth_yolo.py**: the standard 8-class YOLO26n is created, COCO weights are transferred with the same call Ultralytics uses for Run A (612 / 708 tensors; the class branch of the head is re-initialised because the class count changes), and the backbone layers are then swapped. A custom trainer class plugs the model into the standard Ultralytics trainer, including checkpoint resume. One encoder block applies BatchNorm to a globally pooled 1×1 vector, which fails on the last one-image batch of an epoch; those BatchNorm layers fall back to running statistics in that case.

Parameters (unfused)	Run A	Run B	B / A
Backbone	1,365,472	6,634,249 encoder + 136,192 adapters	5.0×
Neck (layers 11–22)	897,152	897,152	1.00
Head (layer 23)	244,296	244,296	1.00
Total	2,506,920	7,911,889	3.16×

Verification before training. Twelve checks gate the full training run, and all passed: backbone output shapes equal YOLO26n's at 640×640 and 224×640; all 232 encoder tensors equal the checkpoint; the encoder inside the detector reproduces the original ZipDepth outputs exactly (max difference 0); all 366 COCO-initialised neck/head tensors equal Run A's starting point; gradients reach 100 % of encoder, adapter and neck tensors; a one-image training step runs; an interrupted run resumes and completes; and the model can overfit 32 training images (Figure 2).

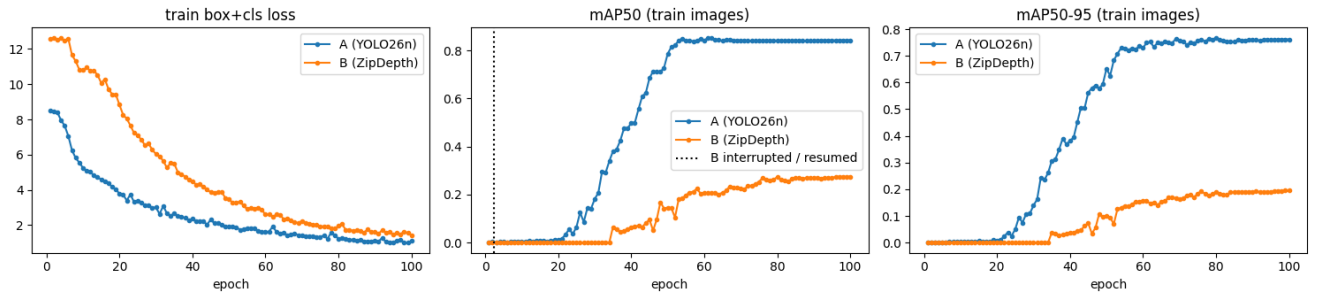


Figure 2. Overfit test on 32 training images (100 epochs, no augmentation, validated on the same images). Run B learns (train loss -92% box, -87% cls; mAP50 0.27) but much more slowly than Run A (mAP50 ≈ 0.84): its adapters start from random weights and its COCO-trained neck receives unfamiliar features. The dotted line marks the deliberate interruption and resume of Run B.

4. Results (tasks 4-5)

4.1 Comparison requested by the task

Metric (val, 1,496 images, 8,128 objects)	A: YOLO26n	B: ZipDepth backbone	B – A	B / A
mAP50-95	0.588	0.472	–0.116	0.80
mAP50	0.849	0.731	–0.118	0.86
Parameters (fused, deployed)	2.38 M	7.14 M	+4.76 M	3.00×
GFLOPs at 640×640 (fused)	5.32	17.82	+12.5	3.35×
Latency, batch 1, 640×640, FP16 (ms)	12.3	11.8	–0.5	0.96×
Throughput, batch 32, 640×640, FP16 (img/s)	513	318	–195	0.62×
Peak GPU memory, training, batch 16 (MiB)	2,423	2,829	+406	1.17×
Peak GPU memory, inference, batch 1 / 32 FP16 (MiB)	102 / 466	131 / 786	+29 / +320	1.29× / 1.69×
Training time per epoch (s)	104	104	+0.6	1.01×

Accuracy from Ultralytics validation of best.pt (conf 0.001, NMS IoU 0.7). Speed: fused network only (no pre-processing, no NMS), 30 warm-up + 200 timed runs, median, CUDA events. Memory: peak allocated by PyTorch. Both models measured back to back in the same session.

4.2 Training behaviour

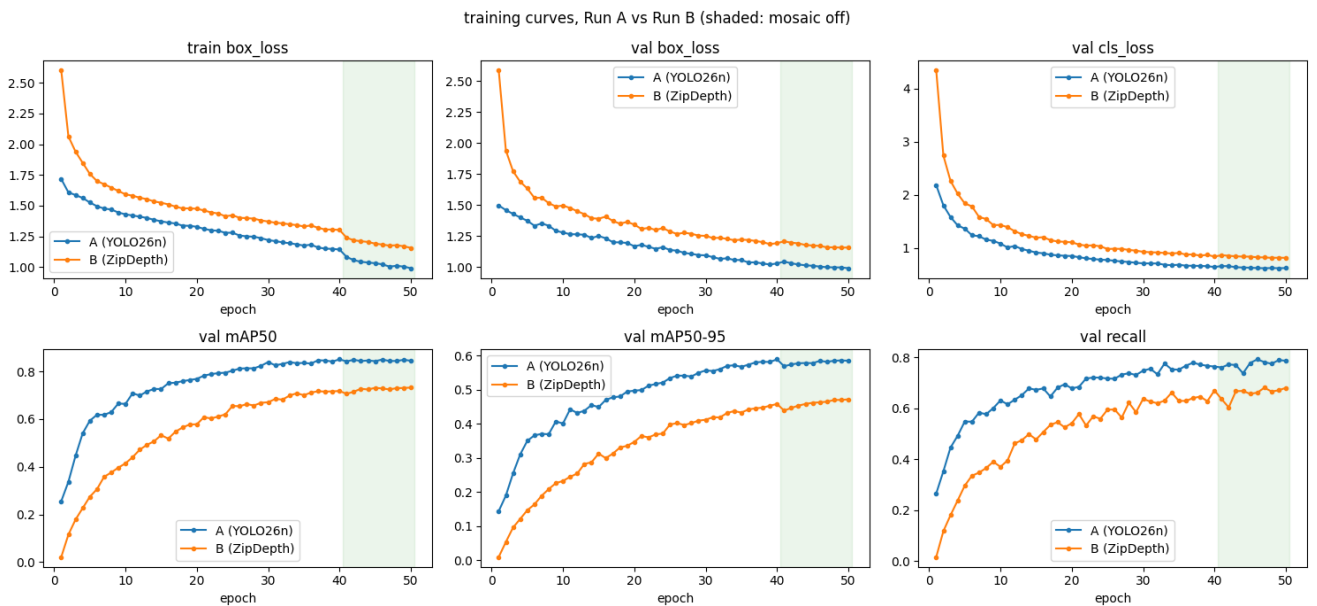


Figure 3. Training curves (shaded: last 10 epochs without mosaic augmentation).

Neither model overfits: validation box loss decreases until the last epoch for both. Run A converges (best epoch 40; mAP50-95 slope over the last five epochs $+0.0005$ / epoch). **Run B does not:** its best epoch is the last one and its slope is $4.5\times$ larger ($+0.0022$ / epoch). The A-B gap narrows from ≈ 0.17 at

epoch 10 to 0.114 at epoch 50, so part of the final gap reflects the 50-epoch budget. Run B also starts from much higher losses (random adapters) and its recall stays lower throughout (≈ 0.10 at the end). Training time is identical because training is limited by data loading on Colab's two CPU cores, not by the GPU.

4.3 Per-class accuracy and error types

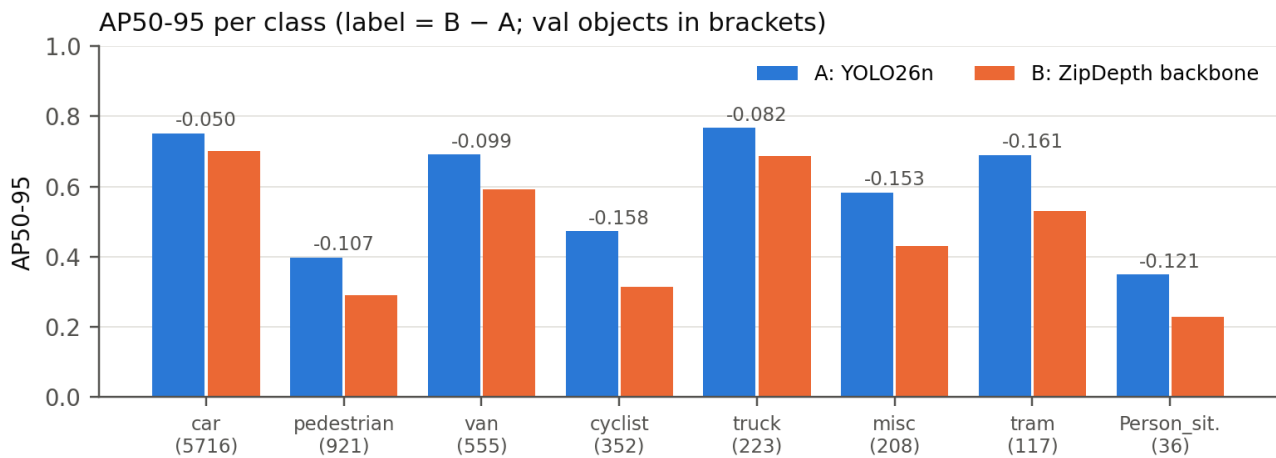


Figure 4. AP50-95 per class. Labels give B – A.

Run B is lower on every class. Cars lose least (-0.050 , -7% relative); the person classes and rare classes lose most (pedestrian -27% , cyclist -33% , Person_sitting -35% , tram -23%). Because mAP averages classes with equal weight, these rare classes dominate the -0.116 headline; weighted by object count the drop is -0.070 .

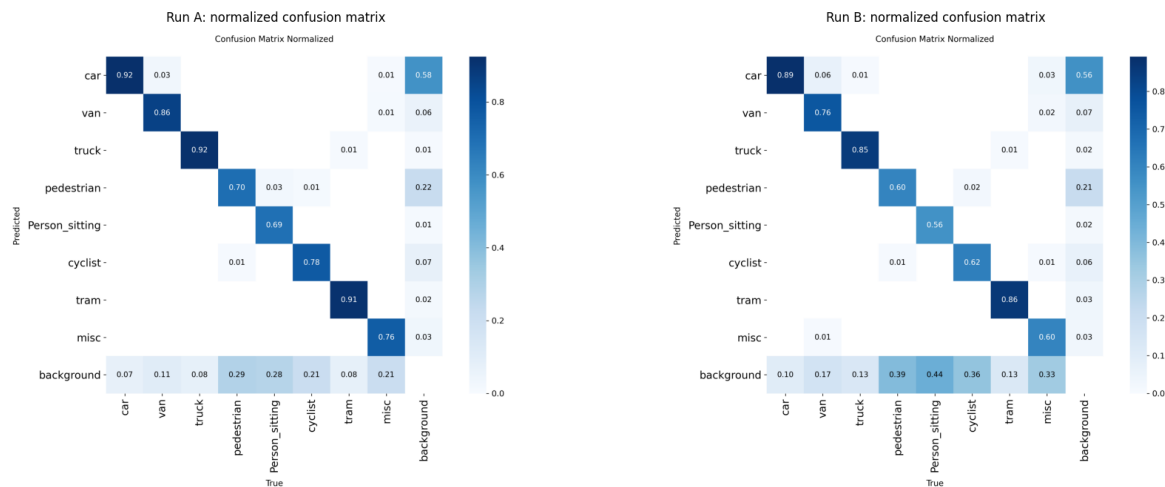


Figure 5. Normalised confusion matrices (columns = true class; last row = missed objects; last column = false positives).

Inter-class confusion stays rare in both models (largest: van $\rightarrow$ car 0.03 in A, 0.06 in B). Almost all of Run B's loss moves to the *background* row, i.e. objects that are not detected at all: pedestrian $0.29 \rightarrow 0.39$, Person_sitting $0.28 \rightarrow 0.44$, cyclist $0.21 \rightarrow 0.36$, misc $0.21 \rightarrow 0.33$. Run B recognises classes but finds fewer objects, consistent with its lower recall ($0.762 \rightarrow 0.679$).

4.4 Accuracy by object size — testing the depth hypothesis

Distant objects appear small, so if depth features help detection the gain should be largest for small boxes. Validation objects are grouped by the height of their ground-truth box in the original image: **small** < 25 px (the KITTI minimum height for the Moderate / Hard levels; 889 objects), **medium** $25\text{--}50$ px (2,903) and **large** ≥ 50 px (4,336). AP is computed per group with the COCO area-range rule: a detection matched to a ground-truth box outside the group, or an unmatched detection whose own height is outside the group, is ignored. With all sizes included, the implementation reproduces Ultralytics' AP exactly (checked on synthetic data). 800 of the 889 small objects are cars, so **car AP50-95 on small objects** is the primary metric.

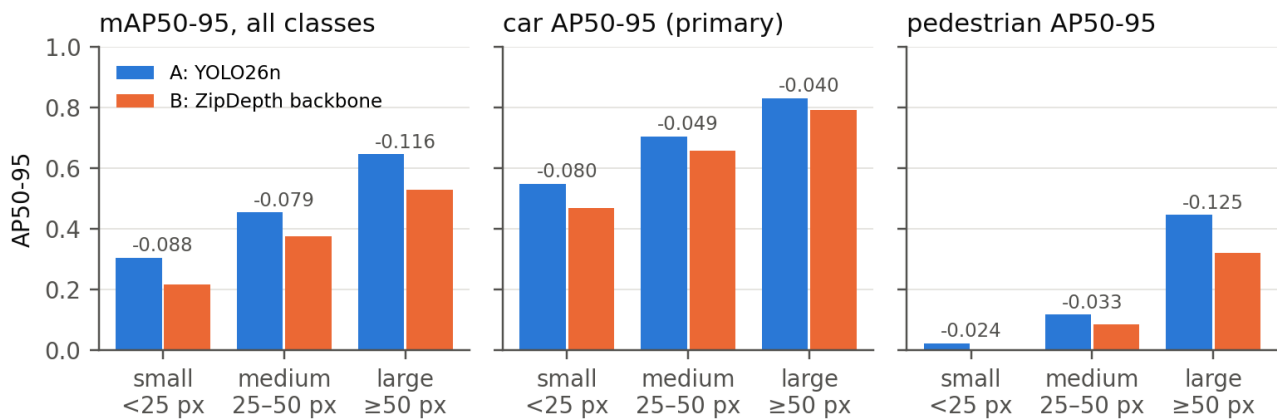


Figure 6. AP50-95 by object size. Small pedestrians (15 objects) are too few for a reliable value.

Run B is lower in every size group, and the relative loss is **largest for small objects**: small cars -15% ($0.550 \rightarrow 0.470$), medium -7% , large -5% . With the recipe used here, the hypothesis that depth-pretrained features help distant objects is **not supported**.

4.5 Cost

Configuration (fused, T4)	A (ms)	B (ms)	B / A
batch 1, 640×640, FP32	11.2	10.5	0.93
batch 1, 640×640, FP16	12.3	11.8	0.96
batch 1, 224×640, FP32 (letterboxed KITTI frame)	13.1	10.4	0.80
batch 1, 224×640, FP16	12.4	16.1	1.30
batch 32, 640×640, FP16 (per batch)	62.4	100.7	1.61

At batch 1 both networks take ≈ 10 – 16 ms regardless of input size or precision, with no consistent ordering: the T4 is idle most of the time and latency reflects the number of kernel launches, not the arithmetic. At batch 32 the GPU is saturated and the real compute cost appears: Run B is $1.61\times$ slower per image. This is well below its $3.35\times$ FLOP ratio because the fused encoder consists mostly of dense 3×3 convolutions that use the GPU efficiently (≈ 5.7 effective TFLOPS vs ≈ 2.7 for YOLO26n's many small depthwise, split/concat and attention operations). On a compute-bound device (embedded GPU or CPU) the batch-1 gap would be much larger; in earlier CPU tests Run B was about $2.2\times$ slower. Training memory grows only 17% because activations, not weights, dominate memory.

4.6 Feature maps

Each panel projects a feature map onto its first three principal components (shown as RGB; the title gives the share of variance they explain) for the validation image with the most small objects. Similar colours mean the network describes those regions similarly. Colours are not comparable between panels; structure and percentages are.

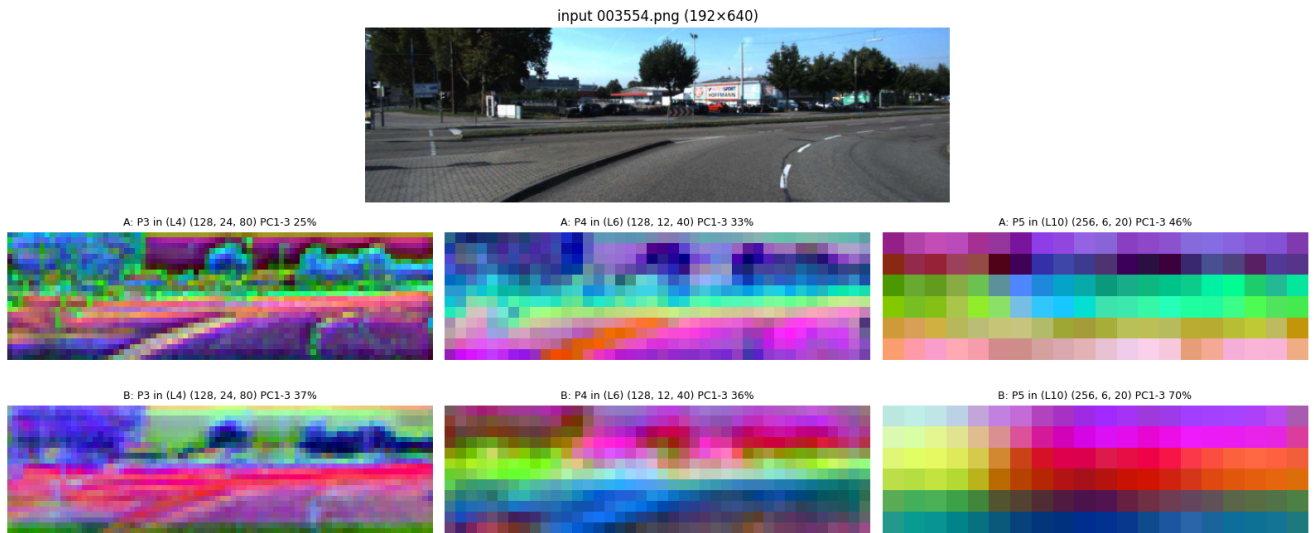


Figure 7. Backbone outputs handed to the neck (layers 4 / 6 / 10) for Run A (row 1) and Run B (row 2), after fine-tuning.

Run A's P3 map keeps local detail (lane markings, pavement texture, sharp tree outlines; 25 % of variance in three components, i.e. high-dimensional). Run B's P3 is smoother and dominated by horizontal bands (37 %). At P5, where both tensors have identical shape, Run B is a smooth top-to-bottom gradient in which three components explain **70 %** of the variance (A: 46 %): even after fine-tuning, the depth encoder describes scene layout and distance rather than object identity.

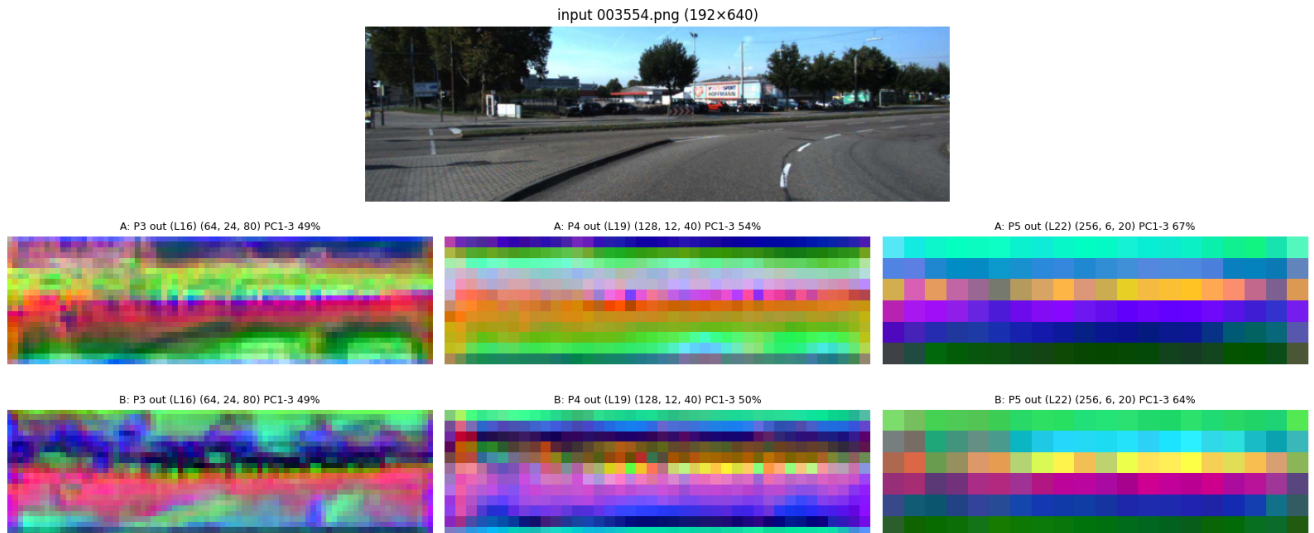


Figure 8. Neck outputs fed to the detection head (layers 16 / 19 / 22), Run A (row 1) and Run B (row 2).

After the neck the two models look alike (49 / 49 %, 54 / 50 %, 67 / 64 %): the fine-tuned neck compensates for most of the difference in its input, but it cannot recreate the local detail missing from Run B's P3, which is consistent with the larger losses on small and thin objects. These maps come from a single image and are indicative only.

5. Discussion

Replacing the YOLO26n backbone with the depth-pretrained ZipDepth encoder made the detector both less accurate and more expensive. The evidence points to the following explanations (not individually tested):

- **Objective mismatch.** Depth is a smooth, low-dimensional quantity; the encoder's features emphasise layout and distance (Figure 7) and separate object classes less well than a detection backbone, which fits the higher miss rate.
- **Interface cost.** The adapters start from random weights and the COCO-trained neck expects YOLO features, so Run B has to re-align three components; it learns more slowly both in the overfit test and in full training.

- **Recipe.** Run B used the baseline recipe unchanged, including lr 0.01 on the pretrained encoder with no freezing or lower encoder learning rate. This keeps the comparison clean but may overwrite useful depth features early in training.
- **Budget.** Run B had not converged after 50 epochs; a longer schedule would narrow the gap, but by an unknown amount.

6. Limitations

- Single seed per run; no confidence intervals.
- No held-out test set (KITTI test labels are not public): the validation split is used both to select best.pt and to report results. This applies to both runs equally, so the comparison remains fair, but absolute numbers are slightly optimistic. Ultralytics' split is a random 80/20 split, so frames of the same drive can occur in both splits.
- DontCare regions are removed from the labels: correct detections of unlabelled distant cars count as false positives.
- Speed was measured on a Tesla T4 only; memory values are peak PyTorch allocations in the benchmark session.
- The contribution of depth pretraining itself is not isolated because Run C has not been trained.

7. Next steps

- **Run C** (same model, random encoder, identical recipe; the code supports it with one argument). $C \approx B$ would mean depth pretraining gives no benefit; $C < B$ would mean it helps, but not enough to beat the detection backbone.
- Train Run B longer (e.g. 100 epochs) to measure the converged gap.
- Protect the pretrained features: freeze the encoder for the first epochs (training adapters, neck and head only), or use a $0.1\times$ learning rate for the encoder.
- Target small objects directly: a higher input resolution or an extra stride-4 level fed from the encoder's stage-1 features.

Appendix A. Qualitative examples



Figure 9. Four validation images fixed in advance (two with the most small objects, two random). Rows per image: ground truth (red = small object), Run A, Run B; detections with confidence ≥ 0.25 . In these car-dominated scenes both models find most objects; Run B tends to assign lower confidences to the same objects (e.g. 001490: van 0.86 $\rightarrow$ 0.74, truck 0.74 $\rightarrow$ 0.46), and some of its extra boxes fall on parked cars that carry no label.

Appendix B. Files and reproducibility

File	Content
baseline_yolo26n_kitti.ipynb	Run A: data table, locked configuration, training (skipped if finished), evaluation, size AP, feature maps, cost
zipdepth_yolo26n_kitti.ipynb	Run B: model construction, 12 sanity checks, training, A-vs-B evaluation, size AP, feature maps, cost, comparison table
zipdepth_yolo.py	ZipDepthBackbone, layer selectors, build_zipdepth_yolo(), ZipDepthTrainer, fusion helper (random encoder: zipdepth_ckpt=None)
lab_eval.py	shared evaluation code: label table, size-bucket AP (validated against Ultralytics), PCA, latency and memory helpers
exp_config.yaml	locked training configuration shared by all runs
results_A.json, results_B.json, comparison_AB.csv	all reported numbers

External code and weights: Ultralytics YOLO26n (yolo26n.pt, COCO); ZipDepth, official repository github.com/fabiotosi92/ZipDepth, checkpoint zipdepth_base.pth; KITTI in Ultralytics YOLO format.